

JAVA DOCS

Java 17–21 Updates

Modern Language Features: Records, Pattern Matching, Sealed Classes & Virtual Threads

Java Agentic AI — Knowledge Base Reference

1. LTS Releases Context

Java 17 (Sep 2021) and Java 21 (Sep 2023) are Long-Term-Support (LTS) releases, meaning extended vendor support well beyond the standard 6-month release cadence. This document consolidates the major features finalized across Java 9 through 21 that are most commonly used and asked about in modern interviews.

2. Records (Java 16+)

Records are immutable data carrier classes that eliminate boilerplate: the compiler auto-generates a canonical constructor, private final fields, public accessors (named after the field, not getX()), equals(), hashCode(), and toString().

```
public record Point(int x, int y) { }

Point p = new Point(3, 4);
p.x();           // 3 - accessor, not getX()
p.equals(new Point(3, 4)); // true - value-based equality auto-generated
System.out.println(p);           // Point[x=3, y=4]

// Compact constructor - add validation without repeating the field list
public record Range(int min, int max) {
    public Range {           // compact constructor
        if (min > max) throw new IllegalArgumentException("min > max");
    }
}
```

Note: Records are implicitly final and cannot extend another class (they implicitly extend `java.lang.Record`), but they can implement interfaces and declare additional static members or methods.

3. Sealed Classes (Java 17)

Sealed classes/interfaces restrict which other classes may extend or implement them, giving exhaustive, compiler-checked control over a type hierarchy.

```
public sealed interface Shape permits Circle, Square, Triangle { }

public final class Circle implements Shape { double radius; }
public final class Square implements Shape { double side; }
public non-sealed class Triangle implements Shape { } // opens this branch back up freely

// Exhaustive switch - no default needed since the compiler knows all permitted subtypes
double area(Shape shape) {
    return switch (shape) {
        case Circle c -> Math.PI * c.radius * c.radius;
        case Square s -> s.side * s.side;
        case Triangle t -> 0.0; // t.computeArea();
    };
}
```

- Permitted subclasses must be final, sealed, or non-sealed (explicitly reopening the hierarchy).

- Enables exhaustive pattern matching in switch without a default branch, since the compiler can verify every case is handled.

4. Pattern Matching for instanceof (Java 16)

```
// Before
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.length());
}

// Pattern matching - cast and bind in one step
if (obj instanceof String s) {
    System.out.println(s.length());    // 's' is in scope, already cast
}
```

5. Pattern Matching for switch (Java 21)

```
Object obj = 42;
String description = switch (obj) {
    case Integer i when i > 100 -> "big integer";    // guarded pattern
    case Integer i -> "small integer: " + i;
    case String s -> "string of length " + s.length();
    case null -> "it's null";
    default -> "something else";
};

// Record pattern deconstruction (Java 21)
record Point(int x, int y) {}
Object o = new Point(3, 4);
if (o instanceof Point(int x, int y)) {
    System.out.println(x + y);    // 7 - fields deconstructed directly
}
```

6. Text Blocks (Java 15)

```
String json = """
    {
        "name": "Claude",
        "role": "assistant"
    }
    """;
// Preserves formatting, no need to escape every quote or concatenate with +
```

7. Virtual Threads (Java 21 — Project Loom)

Virtual threads are lightweight threads managed by the JVM rather than mapped 1:1 to OS threads. They make blocking I/O-style code scale to millions of concurrent tasks without the memory/context-switch overhead of platform threads.

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 100_000; i++) {
        executor.submit(() -> {
            Thread.sleep(Duration.ofSeconds(1)); // blocking call - cheap on a virtual
            thread
            return null;
        });
    }
} // executor auto-closes, awaiting completion
```

Aspect	Platform Thread	Virtual Thread
Mapping	1:1 with an OS thread	Many virtual threads share a small pool of 'carrier' OS threads
Creation cost	Expensive (MBs of stack, OS-managed)	Very cheap (KBs, JVM-managed)
Max practical count	Thousands	Millions
Blocking I/O	Ties up the OS thread	Unmounts from the carrier thread, freeing it for other work

8. Other Notable Additions

Version	Feature
Java 9	Module system (Project Jigsaw / JPMS), JShell REPL, factory methods List.of()/Set.of()/Map.of()
Java 10	Local variable type inference (var)
Java 11	New HttpClient API, String methods (isBlank, strip, repeat, lines)
Java 14	Switch expressions (standardized), helpful NullPointerException messages
Java 16	Records and pattern matching for instanceof (standardized)
Java 17	Sealed classes (standardized)
Java 21	Virtual threads, record patterns, pattern matching for switch (all standardized)

9. Quick Interview Recap

- What problem do records solve? Boilerplate for immutable data carriers — constructor, accessors, equals/hashCode/toString are auto-generated from the field list.
- What do sealed classes enable? Exhaustive, compiler-verified type hierarchies and switch expressions without needing a default branch.
- Why are virtual threads a big deal? They let simple blocking-style code scale to huge levels of concurrency without the cost of one OS thread per task, competing with traditional reactive programming models for I/O-heavy workloads.
- Can a sealed interface have a non-sealed permitted implementation? Yes — non-sealed reopens that specific branch of the hierarchy to unrestricted extension.